Technical Documentation

Complete System Explanation for Engineering Review

Project	RAG-Based Customer Support Assistant
Technology	LangGraph + ChromaDB + Groq LLM + HuggingFace
Document Type	Technical Documentation
Date	April 2025

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines two powerful techniques: information retrieval and text generation. Instead of relying solely on a language model's pre-trained knowledge (which can be outdated or hallucinated), RAG first retrieves relevant documents from a knowledge base and then provides them as context to the LLM for generating a grounded, accurate answer.

Why RAG is Needed

Standard LLMs have a knowledge cutoff and cannot access your private company data. They also hallucinate facts with high confidence. RAG fixes both problems: it retrieves real, up-to-date information from your own documents and grounds the LLM's response in verifiable evidence, dramatically reducing hallucinations.

Use Case: Customer Support Bot

This project builds a customer support assistant for an e-commerce company. The bot reads a product/policy PDF (return policy, shipping info, FAQs) and answers customer queries intelligently. Complex or ambiguous queries are escalated to a human agent via the HITL system.

2. System Architecture Explanation

The system is composed of two distinct pipelines that work together:

Pipeline A — Offline Indexing (runs once):

The PDF is loaded by PyPDFLoader, which extracts raw text page-by-page. RecursiveCharacterTextSplitter then breaks this text into overlapping 500-character chunks. Each chunk is converted to a 384-dimensional embedding vector by the HuggingFace all-MiniLM-L6-v2 model running locally. Finally, ChromaDB stores these vectors in a persistent local directory (./chroma_db) for fast similarity search.

Pipeline B — Online Query Processing (runs per query):

When a user types a question, LangGraph's StateGraph is invoked. The retrieve_node converts the query to a vector and finds the top-3 most similar chunks from ChromaDB. The route_decision function then evaluates confidence (based on context volume and keyword detection). If confident, generate_node calls the Groq LLM with a structured prompt. If not, escalate_node triggers the HITL response.

3. Design Decisions

Chunk Size: 500 chars with 50 overlap

500 characters balances context richness with retrieval precision. Too large chunks reduce specificity; too small chunks lose context. 50-char overlap ensures no sentence is split awkwardly at a boundary.

Embedding: all-MiniLM-L6-v2

This model is lightweight (80MB), runs locally without API keys, and produces high-quality semantic embeddings. It outperforms TF-IDF-based retrieval significantly for natural language queries.

Retrieval: Top-3 chunks (k=3)

Fetching 3 chunks provides enough context without overloading the LLM prompt. More chunks increase latency and token usage; fewer may miss critical information.

Prompt Design: Context-anchored instruction

The prompt explicitly instructs the LLM to use ONLY the provided context and to say 'I don't know' if the answer isn't there. This prevents hallucination and keeps responses grounded.

LLM: Groq LLaMA-3.1-8b-instant

Free API with fast inference. 8B parameter model is sufficient for Q&A; tasks. Groq's hardware acceleration gives sub-second response times, making the CLI experience smooth.

4. LangGraph Workflow Explanation

LangGraph is a graph-based orchestration framework built on top of LangChain. It allows defining AI workflows as directed graphs where each node is a Python function that reads and modifies a shared state object.

Node	Input State Fields	Output State Fields	Responsibility
retrieve_node	query	context, confidence	Fetches relevant chunks from ChromaDB
route_decision	context, confidence, query	Returns routing string	Decides generate vs escalate
generate_node	query, context	answer	Calls LLM with formatted prompt
escalate_node	query	answer	Creates HITL escalation message

5. Conditional Logic & Intent Detection

The routing layer uses two signals to determine the appropriate response path:

Signal 1 — Context Volume: After retrieval, if the joined context string is fewer than 200 characters, the system assumes the PDF does not contain relevant information. Confidence is set to 0.3, triggering escalation.

Signal 2 — Keyword Detection: The query is scanned for keywords indicating urgency or legal/financial complexity: 'urgent', 'legal', 'lawsuit', 'refund escalate'. These are hard-routed to the escalation path regardless of confidence.

6. HITL Implementation

Role of Human Intervention: HITL ensures that high-stakes, ambiguous, or context-poor queries are not answered incorrectly by the LLM. Instead, a human agent reviews the query and provides a verified response.

Current Implementation: The escalate_node prints a structured message to the CLI indicating escalation, logging the original query. In a production system, this would integrate with Zendesk, Jira

Service Desk, or send an email to the support team.

Benefits: Prevents incorrect LLM answers for edge cases. Builds user trust. Provides a safety net for legal/financial queries.

Limitations: Current CLI implementation is simulated — no real ticket system integration. Keyword-based routing may miss some escalation scenarios.

7. Challenges & Trade-offs

Challenge	Trade-off	Decision
Retrieval accuracy vs speed	Larger embedding models are more accurate but slower	Chose all-MiniLM-L6-v2 — best balance for CLI use
Chunk size vs context quality	Larger chunks = more context but less precise retrieval	500 chars with 50 overlap works well for policy documents
Cost vs performance	GPT-4 is better but expensive; smaller models are free but weaker	Groq Llama 3.1 8b instant is free and sufficient for Q&A
Local vs cloud vector DB	ChromaDB is local and free; Pinecone scales better	ChromaDB chosen for zero-cost internship deployment

8. Testing Strategy

Testing was performed manually using representative queries across three categories:

Test Category	Sample Query	Expected Result	Actual Result
Normal RAG query	What is the return policy?	Structured return policy from PDF	PASS — returned 7-day policy details
Out-of-scope query	What are working hours?	'I don't know' response	PASS — correctly said not in PDF
HITL trigger (keyword)	legal refund escalate	HITL escalation message	PASS — escalated to human agent
HITL trigger (urgency)	I need urgent help with my account	HITL escalation message	PASS — escalated correctly
Empty context	Random unrelated question	Escalation or I don't know	PASS — low confidence escalated

9. Future Enhancements

Multi-Document Support: Replace PyPDFLoader with DirectoryLoader to ingest entire folders of PDFs. Use metadata filtering in ChromaDB to scope search per document category.

Feedback Loop: After human resolves an escalated query, add the Q&A; pair back into ChromaDB as a new knowledge chunk, continuously improving the bot.

Memory Integration: Add ConversationBufferMemory to maintain multi-turn dialogue context, enabling follow-up questions without repeating context.

Web Interface: Replace CLI with a Streamlit or FastAPI web UI for browser-based interaction and file upload support.

Production Deployment: Containerize with Docker, deploy ChromaDB as a managed service (Pinecone/Weaviate), and expose via REST API behind an authentication layer.

Evaluation Metrics: Implement RAGAS (Retrieval-Augmented Generation Assessment) metrics: faithfulness, answer relevancy, and context recall scores.